

RII Weight Learner: Complete Documentation

Implementation Documentation

December 4, 2025

Contents

1	Overview	2
2	Mathematical Foundation	2
2.1	RII Score Formula	2
2.2	Loss Function	2
2.3	Gradient Update	2
3	Class: RIIWeightLearner	3
3.1	Constructor	3
3.2	Parameters	3
3.3	Attributes	3
4	Methods	3
4.1	fit(X, verbose=True)	3
4.2	compute_rii(X)	4
4.3	get_feature_importance(feature_names=None)	4
5	Usage Example	4
5.1	Basic Usage	4
5.2	Complete Workflow	5
6	Scaling to Large Datasets	5
6.1	Memory Optimization	6
7	Hyperparameter Tuning	6
7.1	Learning Rate	6
7.2	Number of Clusters	6
7.3	Batch Size	6
8	Feature Engineering	7
8.1	Recommended Features	7
8.2	Feature Preprocessing	7
9	Troubleshooting	7
9.1	Weights Don't Change	7
9.2	Training Loss Increases	8
9.3	Memory Error	8
10	API Reference Summary	8
11	Conclusion	8

1 Overview

The **RII (Research Impact Index) Weight Learner** is an unsupervised machine learning framework for computing weighted impact scores for scientific articles. It learns optimal feature weights by minimizing intra-cluster variance of RII scores, eliminating the need for labeled data.

2 Mathematical Foundation

2.1 RII Score Formula

For article i , the RII score is computed as:

$$RII_i = \sum_{j=1}^F w_j \cdot x_{i,j}$$

where:

- $x_{i,j}$ is the value of feature j for article i
- w_j is the learned weight for feature j
- F is the total number of features

2.2 Loss Function

The training minimizes the total intra-cluster variance:

$$\mathcal{L}(w) = \sum_{c \in C} \text{Var}_c$$

where for each cluster c :

$$\text{Var}_c = \frac{1}{|c|} \sum_{i \in c} (RII_i - \overline{RII}_c)^2$$

and

$$\overline{RII}_c = \frac{1}{|c|} \sum_{i \in c} RII_i$$

2.3 Gradient Update

Weights are updated using gradient descent:

$$\frac{\partial \text{Var}_c}{\partial w_j} = \frac{2}{|c|} \sum_{i \in c} (RII_i - \overline{RII}_c) (x_{i,j} - \bar{x}_{c,j})$$

The weight update rule is:

$$w_j \leftarrow w_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_j}$$

After each update, weights are normalized to sum to 1:

$$w_j := \frac{w_j}{\sum_k w_k}$$

3 Class: RIIWeightLearner

3.1 Constructor

```
1 RIIWeightLearner(  
2     n_clusters=10,  
3     learning_rate=0.01,  
4     max_iterations=100,  
5     batch_size=10000,  
6     random_state=42,  
7     non_negative=True  
8 )
```

3.2 Parameters

Parameter	Type	Default	Description
n_clusters	int	10	Number of clusters for Mini-Batch K-Means
learning_rate	float	0.01	Learning rate (η) for gradient descent
max_iterations	int	100	Maximum number of training iterations
batch_size	int	10000	Size of mini-batches for training
random_state	int	42	Random seed for reproducibility
non_negative	bool	True	Enforce non-negative weights

Table 1: Constructor parameters

3.3 Attributes

- **weights**: Learned feature weights (ndarray)
- **scaler**: Fitted StandardScaler for normalization
- **clusterer**: Fitted MiniBatchKMeans model
- **loss_history**: Training loss at each iteration (list)

4 Methods

4.1 fit(X, verbose=True)

Learn RII weights from article data using mini-batch training.

Parameters:

- **X**: array-like, shape ($n_{samples}, n_{features}$) - Feature matrix
- **verbose**: bool, default=True - Print training progress

Returns:

- **self**: Returns self for method chaining

Training Process:

1. Standardize features using StandardScaler
2. Initialize weights uniformly ($1/n_{features}$)
3. Perform initial clustering
4. For each iteration:
 - Sample a mini-batch
 - Predict cluster assignments
 - Compute loss (intra-cluster variance)
 - Compute gradients
 - Update weights via gradient descent
 - Normalize weights
 - Periodically update clustering model

4.2 compute_rii(X)

Compute RII scores for new or existing data.

Parameters:

- **X**: array-like, shape $(n_{samples}, n_{features})$ - Feature matrix

Returns:

- **rii_scores**: ndarray, shape $(n_{samples},)$ - RII score for each article

4.3 get_feature_importance(feature_names=None)

Get feature importance based on learned weights.

Parameters:

- **feature_names**: list of str, optional - Names of features

Returns:

- **importance_df**: DataFrame with 'Feature' and 'Weight' columns

5 Usage Example

5.1 Basic Usage

```

1 import numpy as np
2 from rii_weight_learner import RIIWeightLearner
3
4 # Load your article data
5 X = load_article_features() # shape: (n_articles, n_features)
6
7 # Initialize learner
8 learner = RIIWeightLearner(
9     n_clusters=15,
10    learning_rate=0.01,
11    max_iterations=100,
12    batch_size=10000
13 )

```

```

14
15 # Train the model
16 learner.fit(X, verbose=True)
17
18 # Compute RII scores
19 rii_scores = learner.compute_rii(X)
20
21 # Get feature importance
22 feature_names = ['citations', 'num_authors', 'revenue',
23                  'years_since_pub', 'h_index']
24 importance = learner.get_feature_importance(feature_names)
25 print(importance)

```

5.2 Complete Workflow

```

1 import numpy as np
2 import pandas as pd
3
4 # 1. Prepare data
5 df = pd.read_csv('articles.csv')
6 feature_names = ['citations', 'num_authors', 'revenue',
7                  'years_since_pub', 'avg_author_hindex']
8 X = df[feature_names].values
9
10 # 2. Train model
11 learner = RIIWeightLearner(n_clusters=15)
12 learner.fit(X, verbose=True)
13
14 # 3. Get results
15 rii_scores = learner.compute_rii(X)
16 df['RII_score'] = rii_scores
17
18 # 4. Analyze
19 print(f"Mean RII: {np.mean(rii_scores):.4f}")
20 print(f"Std RII: {np.std(rii_scores):.4f}")
21
22 # Top 10 articles
23 top_articles = df.nlargest(10, 'RII_score')
24 print(top_articles[['title', 'RII_score']])

```

6 Scaling to Large Datasets

For datasets with 10 million articles, use these recommended settings:

```

1 learner = RIIWeightLearner(
2     n_clusters=30,          # More clusters
3     learning_rate=0.005,    # Lower learning rate
4     max_iterations=200,     # More iterations
5     batch_size=50000,       # Larger batches
6     random_state=42,
7     non_negative=True
8 )
9
10 learner.fit(X_10M, verbose=True)

```

6.1 Memory Optimization

For datasets that don't fit in memory:

```
1 # Process in chunks
2 chunk_size = 100000
3 n_chunks = len(df) // chunk_size + 1
4
5 for i in range(n_chunks):
6     start_idx = i * chunk_size
7     end_idx = min((i + 1) * chunk_size, len(df))
8
9     X_chunk = df.iloc[start_idx:end_idx][features].values
10    rii_chunk = learner.compute_rii(X_chunk)
11
12    df.loc[start_idx:end_idx, 'RII_score'] = rii_chunk
```

7 Hyperparameter Tuning

7.1 Learning Rate

- **Too high:** Training unstable, loss oscillates
- **Too low:** Very slow convergence
- **Recommended:** Start with 0.01, adjust based on loss curve
- **Range:** 0.001 - 0.1

7.2 Number of Clusters

Use the elbow method to determine optimal K :

```
1 from sklearn.cluster import MiniBatchKMeans
2 from sklearn.preprocessing import StandardScaler
3
4 # Sample for elbow method
5 X_sample = X[np.random.choice(len(X), 50000)]
6 scaler = StandardScaler()
7 X_scaled = scaler.fit_transform(X_sample)
8
9 # Test different K values
10 inertias = []
11 for k in range(5, 51, 5):
12     kmeans = MiniBatchKMeans(n_clusters=k)
13     kmeans.fit(X_scaled)
14     inertias.append(kmeans.inertia_)
15
16 # Choose K from elbow point
```

7.3 Batch Size

- **Larger batches:** Better gradient estimates, slower
- **Smaller batches:** Faster iterations, more noise
- **Recommended:** 0.1% - 1% of dataset size
- **Range:** 1000 - 50000

8 Feature Engineering

8.1 Recommended Features

1. Citation-based:

- Total citations
- Citations per year
- Citation velocity

2. Author-based:

- Number of authors
- Average h-index of authors
- Maximum h-index among authors

3. Publication-based:

- Journal impact factor
- Years since publication
- Publication type

4. Financial:

- Research revenue/funding
- Grant amounts

8.2 Feature Preprocessing

```
1 # Log-transform skewed features
2 X[:, 0] = np.log1p(X[:, 0]) # citations
3 X[:, 2] = np.log1p(X[:, 2]) # revenue
4
5 # Handle missing values
6 X = np.nan_to_num(X, nan=0.0)
7
8 # Remove outliers
9 from scipy import stats
10 z_scores = np.abs(stats.zscore(X, axis=0))
11 X = X[(z_scores < 3).all(axis=1)]
```

9 Troubleshooting

9.1 Weights Don't Change

Solutions:

- Increase learning rate (0.01 $\rightarrow$ 0.05)
- Check feature distributions
- Ensure proper standardization
- Remove constant features

9.2 Training Loss Increases

Solutions:

- Decrease learning rate ($0.01 \rightarrow 0.005$)
- Check for NaN/Inf values
- Ensure features are scaled properly

9.3 Memory Error

Solutions:

- Reduce batch_size
- Process data in chunks
- Use smaller sample for initial clustering
- Use float32 instead of float64

10 API Reference Summary

Method	Purpose	Input	Output
<code>__init__()</code>	Initialize learner	Hyperparameters	Object
<code>fit(X)</code>	Train model	Feature matrix	self
<code>compute_rii(X)</code>	Get RII scores	Feature matrix	RII scores
<code>get_feature_importance()</code>	Analyze weights	Feature names	DataFrame

Table 2: API methods summary

11 Conclusion

The RII Weight Learner provides a scalable, unsupervised approach to learning article impact scores. Key advantages:

- No labeled data required
- Scales to millions of articles
- Data-driven weight learning
- Interpretable feature importance
- Modern alternative to h-index